

Code review — david-recapitulare-matrici

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 8 — 2026-09-29 — sha d44bbe9 („finish tema”, peste 3d4125b)

Runda asta are o veste foarte bună și una foarte proastă, și sunt legate.

Vestea bună: ai reparat compilarea lui `functii.h` și ai atacat fișa 3. Cele trei funcții noi din `functii.h` — `nrChenare`, `nrElementeChenar`, `maximChenar` — sunt **corecte, toate trei, pe toate valorile din fișă**. Le-am rulat una câte una:

Cerință	Testat pe	Ieșire	Fișa cere
a) <code>nrChenare</code>	6×6, 5×5, 5×3, 4×4, 1×4	3, 3, 2, 2, 1	3, 3, 2, 2, 1 ✓
b) <code>nrElementeChenar</code>	A k=0,1,2,3 · B k=1	20, 12, 4, -1 · 3	idem ✓
c) <code>maximChenar</code>	A k=0,1,2	92, 88, 75	idem ✓

Iar `nrChenare` e scrisă cu $(\text{minim}+1)/2$, adică **exact formula dedusă**, nu una ghicită. Și `nrElementeChenar` are gărzile pentru chenarul degenerat (`if(i1==i2)`, `if(j1==j2)`) și întoarce `-1` când chenarul nu există. Asta e prima dată în opt runde când scrii singur gărzile despre care vorbim din runda 5. **Ai regula.**

Vestea proastă: în aceeași zi în care ai scris gărzile corect într-o funcție nouă, **le-ai șters din `sumaChenar`**, unde le aveai deja. Și `solutii.h`, fișierul în care ai scris cele patru soluții noi, are **16 erori de compilare** pe care n-ai cum să le vezi, pentru că nu ajunge la compilator.

Începem de acolo.

🔴 B18 — `solutii.h` are 16 erori și nu ajunge niciodată la compilator

`main.cpp:2`

```
#include "functii.h"
```

Ai mutat include-ul de pe `fisa2.h` pe `functii.h` — bine, `functii.h` se compilează acum. Dar codul pe care l-ai scris azi nu e în `functii.h`, e în `solutii.h`. Iar `solutii.h` nu e inclus de nicăieri:

Fișier	Linii	Inclus de	Ajunge la compilator
functii.h	777	main.cpp:2	DA
solutii.h	559	nimeni	NU
exercitiiclasa.h	132	nimeni	NU
fisa2.h	74	nimeni	NU

Deci F9 îți scrie din nou „0 errors” — dar de data asta pe 777 de linii din 1542. L-am compilat eu, separat:

```
g++ -std=c++11 -fsyntax-only -x c++ solutii.h
16 errors generated.
```

DE CE. Compilatorul nu vede foldere, vede **un singur fișier**: cel dat în proiect ca sursă (`main.cpp`). Tot restul ajunge la el numai prin lanțul de `#include`, ca un fir. Firul tău e acum `main.cpp → functii.h` și se oprește acolo. Un `.h` care nu e pe fir nu e „cod care așteaptă” — pentru compilator **nu există**, la fel ca un fișier de pe alt disc.

Că apare în panoul din stânga al Code::Blocks nu schimbă nimic: panoul e o listă de fișiere pe care le poți deschide, nu o listă de fișiere care se compilează. Asta e a doua oară la rând când „0 errors” înseamnă altceva decât crezi (runda 7: `main.cpp` includea doar `fisa2.h`). Nu e o greșeală nouă, e **aceeași greșeală mutată un fișier mai încolo**.

`solutii.h` are deja `#include "functii.h"` pe linia 3. Deci firul corect e mai scurt decât crezi:

FIX — `main.cpp` :

```
#include "solutii.h"

int main()
{
    solutieA();
}
```

Dai F9. Vezi cele 16 erori. **Abia atunci** treci la B20 și B21, care sunt fix ele.

🔴 B19 — REGRESIE: ai șters gărzile din `sumaChenar`, și acoladele goale au rămas

`functii.h:572` și `functii.h:578`

La `74eafb9` funcția arăta așa:

```

if(i1!=i2)
{
    for(int j=j2-1;j>=j1;j--){
        s+=a[i2][j];
    }
}
if(j1!=j2)
{
    for(int i=i2-1;i>i1;i--){
        s+=a[i][j1];
    }
}

```

La `d44bbe9` arată așa:

```

{
    for(int j=j2-1;j>=j1;j--){
        s+=a[i2][j];
    }
}

{
    for(int i=i2-1;i>i1;i--){
        s+=a[i][j1];
    }
}

```

Acoladele alea două care atârnă fără nimic în fața lor sunt **urma gărzilor pe care le-ai șters**. Când ai reparat eroarea de compilare `for(int j=j1; ...)` de pe linia 563, ai șters și cele două `if` -uri de deasupra, dar ai lăsat corpurile lor.

Ce face asta, rulat:

```

sumaChenar(B,5,3,0) = 96    corect
sumaChenar(B,5,3,1) = 32    corect ar fi 24
total                = 128   corect ar fi 120

```

B e matricea 5×3 din fișa 3. Suma tuturor elementelor ei e 1+2+...+15 = **120**. Tu obții **128**. Diferența e 8, adică `a[2][1]` numărat de două ori.

DE CE. Chenarul 1 al unei matrice 5×3 are `i1=1, j1=1, i2=3, j2=1` — o **singură coloană**: elementele 5, 8, 11. Funcția ta merge pe patru laturi. Prima (`j` de la 1 la 1) ia 5. A doua (`i` de la 2 la 3) ia 8 și 11. Latura a treia nu rulează. **Latura a patra** (`i` de la `i2-1=2` în jos, cât timp `i>i1=1`) mai ia o dată `a[2][1] = 8`.

Când chenarul e un dreptunghi adevărat, cele patru laturi sunt distincte. Când degenerază într-o linie sau într-o coloană, latura de întoarcere **calcă peste cea de ducere**. Gărzile `if(i1!=i2)` și `if(j1!=j2)` exact asta întreabă: *mai*

am o a doua latură, sau e aceeași?

De aceea nu s-a văzut nimic pe matricea A: A e 6×6, niciun chenar al ei nu degenerază, și pe A îți dă 874 / 627 / 197 — corect. **Bug-ul nu apare pe datele pe care le testezi tu.** Fișa 3 îți dă matricea B tocmai ca să iasă la suprafață.

Și e aceeași regulă pe care ai scris-o corect azi în `nrElementeChenar` (`functii.h:738,741`). Ai scris gărzile într-un loc și le-ai șters din altul, în același commit.

FIX: pune înapoi `if(i1!=i2)` înaintea acoladei de la `:572` și `if(j1!=j2)` înaintea celei de la `:578` . Apoi rulează `sumaChenar(B,5,3,0) + sumaChenar(B,5,3,1)` și verifică să dea **120**.

🔴 B20 — trei funcții noi folosesc `a`, `n`, `m` ca și cum ar fi globale

`solutii.h:487`, `:538`, `:542`

Zece din cele 16 erori sunt astea. Trei cazuri, aceeași cauză:

solutiiC — `solutii.h:486–497` : folosești `n` și `m` la linia 487, dar le declari la 496–497.

```
int nrChenare;
if(n<m)           // n si m nu exista inca
    nrChenare=n/2;

...

int n=5;          // abia aici apar
int m=5;
```

solutieC — `solutii.h:533–539` : citești doar `k`, dar apelezi `maximChenar(a,n,m,k)` . `a`, `n` și `m` nu sunt declarate nicăieri în funcție.

solutieD (cea nouă) — `solutii.h:540–558` : la fel, `nrChenare(n,m)` și `sumaChenar(a,n,m,0)` cu trei variabile inexistente.

DE CE. În C++ un nume e vizibil **de la linia pe care e declarat în jos**, până la acolada care închide blocul. Compilatorul citește fișierul o singură dată, de sus în jos: când ajunge la `if(n<m)` și n-a întâlnit încă niciun `n`, nu are ce căuta mai departe — raportează `use of undeclared identifier` pe loc.

În `solutieA` și `solutieB` ai făcut-o corect: `int n,m;` apoi `cin>>n>>m;` apoi le folosești. În celelalte trei ai scris corpul funcției copiind din altă funcție și ai uitat capul — matricea și dimensiunile. Ele nu se moștenesc de la funcția vecină: fiecare funcție pornește cu tabla goală.

FIX: în `solutieC` și în `solutieD` declară matricea și dimensiunile la începutul funcției, ca în `solutieChenarB` (`solutii.h:368–377`). În `solutiiC`, mută cele două linii `int n=5;` `int m=5;` deasupra lui `if(n<m)` .

● B21 — `solutieD` e scrisă de două ori, în același fișier

`solutii.h:393` și `solutii.h:540`

```
solutii.h:540:6: error: redefinition of 'solutieD'
solutii.h:393:6: note: previous definition is here
```

Prima `solutieD` (`:393`) e cerința d) din **fișa 2** — ștergerea coloanelor palindrom. A doua (`:540`) e cerința d) din **fișa 3** — chenarul cu suma maximă. Sunt două probleme diferite cu același nume.

Și mai e ceva: `solutiiC` (`:479`) e o **primă încercare abandonată** la aceeași cerință d) din fișa 3 — aceeași căutare de maxim peste chenare, scrisă înainte, cu formula veche $n/2$ și fără afișare la final. Ai rescris-o mai jos ca `solutieD` și ai lăsat-o pe prima acolo.

DE CE. Regula e simplă: într-un program, un nume de funcție poate aparține **unei singure** funcții. Nu contează că sunt în locuri diferite din fișier — ajung în același program, iar la legare compilatorul n-ar ști pe care s-o cheme. (Header guard-ul `#ifndef SOLUTII_H_INCLUDED` nu te ajută: el oprește *includerea aceluiași fișier de două ori*, nu *scrierea aceleiași funcții de două ori*. Ți-am scris asta și în runda 7, la M13 — atunci erau `stergereLinie` și `stergereColoana`.)

Problema de fond e că ai două fișe în același fișier și numerotarea lor se ciocnește. Fișa 2 are a)–l), fișa 3 are a)–h).

FIX: dă-le nume care spun din ce fișă sunt — `solutieF3A`, `solutieF3B`, `solutieF3C`, `solutieF3D`. Și șterge `solutiiC`: e o ciornă, `solutieD` cea nouă o înlocuiește complet.

● B22 — a treia oară $n/2$, în aceeași zi în care ai scris formula corectă

`solutii.h:487` (nou, azi) · `solutii.h:381` (runda 7, neatins) · `fisa2.h:66` (runda 7, neatins)

Asta e constatarea care mă pune pe gânduri cel mai tare din toată runda.

Azi ai scris, în `functii.h:716`, formula **corectă**, dedusă:

```
int nrChenare(int n,int m)
{
    int minim;
    if(n<m) minim=n; else minim=m;
    return (minim+1)/2;
}
```

Și în același commit, în `solutii.h:486–493`, ai scris **a treia oară** varianta greșită:

```
int nrChenare;
if(n<m) nrChenare=n/2;
else    nrChenare=m/2;
```

`min/2` pierde chenarul degenerat din mijloc: pentru 3×3 dă 1, deși chenarul de grad 1 există (un singur element, `a[1][1]`). `(min+1)/2` dă 2, corect. **Plus-unu-ul e exact chenarul pe care celălalt îl pierde** — de-aia e acolo.

DE CE se întâmplă asta. Nu e o problemă de înțelegere: formula corectă e scrisă de mână ta, în același commit, la 230 de linii distanță. E o problemă de **unde te uiți când scrii**. Când ai scris `solutiiC` te-ai uitat în `solutieChenarB`, funcția de deasupra, și ai copiat de acolo — inclusiv formula veche. N-ai apelat funcția pe care tocmai o scrisesseși, ai copiat o bucată de cod vecin.

E același lucru cu B9 (ai copiat buclele din `afisareChenar` în `algoritmSpirala` în loc s-o apelezi) și cu M13 din runda 7 (ai rescris patru funcții pe care le aveai). De fiecare dată: **copiezi textul în loc să chemi funcția**, iar copia rămâne în urmă când originalul se corectează. O funcție reparată o dată e reparată peste tot; un text copiat de trei ori trebuie reparat de trei ori.

FIX: în toate cele trei locuri, apelează `nrChenare(n,m)` :

```
int nr = nrChenare(n,m);
if(k>=nr) cout<<"NU EXISTA";
```

iar în `fisa2.h:66` :

```
for(int i=0;i<nrChenare(n,m);i++)
    cout<<sumaChenar(a,n,m,i)<<endl;
```

(În `solutieD` cea nouă, `solutii.h:542`, ai făcut-o deja corect. Acolo apelezi funcția.)

B23 — `isPrim` nu testează niciodată împărțirea la 2

`exercitiiclasa.h:100`

```
int isPrim(int n )
{
    if(n<2) return 0;
    for(int d=3; d*d<=n; d+=2)
        if(n%d==0) return 0;
    return 1;
}
```

Bucula pornește de la `d=3` și sare din 2 în 2, deci încearcă 3, 5, 7, 9... **2 nu e încercat niciodată**. Rulat:

```
isPrim: 4 6 8 14 16 22 26 32 34 38 -> toate intorc 1 (prim)
```

Iar `i_prim` (:114), care o folosește, se oprește pe primul număr par:

```
i_prim(10) = 0    corect ar fi 4    (7 si 11)
i_prim(20) = 0    corect ar fi 4    (19 si 23)
```

DE CE. Tiparul „sar peste pare, merg din 2 în 2” e corect, dar el presupune că **ai scos deja cazul 2 din joc** înainte de buclă. Forma completă are două jumătăți, și tu ai scris-o doar pe a doua:

```
if(n<2) return 0;
if(n%2==0) return n==2;
for(int d=3; d*d<=n; d+=2)
    ...
```

Fără prima linie, optimizarea „sar peste pare” devine „nu verific niciodată pare” — exact opusul. Optimizarea taie o verificare pe care presupune că ai făcut-o altundeva; dacă n-ai făcut-o, taie răspunsul.

Și mai e o consecință: `i_prim` cu un `n` pentru care nu găsește niciun prim în jos ar coborî la infinit (`isPrim` întoarce 0 pentru orice număr negativ, iar `p1--` nu se oprește niciodată). Acum nu se vede, tocmai fiindcă funcția zice „prim” mult prea des.

FIX: adaugă `if(n%2==0) return n==2;` înainte de buclă. Apoi rulează `isPrim` pe 2, 3, 4, 5, 8, 9, 100 — trebuie să iasă `1 1 0 1 0 0 0`.

Și încă ceva: ai acum **trei** funcții de primalitate în proiect. `isNumarPrim` (`functii.h:223`) e corectă și e singura folosită efectiv. `nrPrim` (`functii.h:588`) e stricată (vezi B15) și nu o apelează nimeni. `isPrim` (`exercitiiclasa.h:96`) e stricată. Vezi M16.

🟡 M16 — trei funcții care răspund la aceeași întrebare, două stricate

Funcția	Unde	Stare	Apelată de
<code>isNumarPrim</code>	<code>functii.h:223</code>	corectă	<code>primeChenar</code> , <code>solutii.h:110</code>
<code>nrPrim</code>	<code>functii.h:588</code>	stricată (B15)	nimeni
<code>isPrim</code>	<code>exercitiiclasa.h:96</code>	stricată (B23)	<code>i_prim</code>

`nrPrim` e cod mort **și** greșit: cea mai proastă combinație, fiindcă stă acolo și pare disponibilă. Șterge-o. Iar în `exercitiiclasa.h` , după ce repari `isPrim` , gândește-te dacă n-ar trebui să includă `functii.h` și să folosească `isNumarPrim` — ai deja funcția.

🟡 M17 — exercitiiclasa.h nu e inclus de nicăieri (132 de linii netestate)

Fișierul din lecția de la clasă (`3d4125b`) a fost comis, dar nu e pe firul de `#include` . Se compilează curat singur — l-am verificat — dar nimic din el n-a rulat vreodată la tine. B23 e dovada: un bug care s-ar fi văzut din prima rulare a stat neobservat șase zile.

Aceeași soluție ca la B18: un `#include` și un apel în `main` .

🟡 M18 — `int p=pow(10,nr-1)` — `pow` lucrează cu numere reale

`exercitiiclasa.h:39` , în `mutaPrimaCif`

`pow` întoarce un `double` . Pentru puteri mici e exact, dar de pe la `pow(10,15)` în sus rezultatul poate fi `999999999999999.9` , iar conversia la `int` **taie**, nu rotunjește — ai primi o putere mai mică cu 1. Pe numerele din exercițiu funcționează (am verificat: `mutaPrimaCif(1234)` = 4123, `mutaPrimaCif(51234)` = 51234), dar obiceiul e prost.

Puterea de 10 se calculează cu înmulțiri, în întregi:

```
int p=1;
for(int i=1;i<nr;i++) p=p*10;
```

🟢 Cleanups

C21 — `doxygen/html/` (62 de fișiere) e comis în repo. Sunt fișiere *generate* de Doxygen din codul tău: `jquery.js` , `doxygen.css` , 20 de `.png` , paginile HTML. Se refac oricând dintr-o singură comandă. Regula e aceeași ca la `bin/` și `obj/` : **ce poate fi regenerat nu se comite**. Altfel, la fiecare rulare de Doxygen ai un commit cu mii de linii schimbate în care nu se mai vede ce cod ai scris tu. Păstrează `doxygen/doxyfile` (ăla e configurația, e scrisă de tine), scoate `doxygen/html/` .

C20 (din runda 7, neatins) — tot nu există `.gitignore` . Acum e mai important, fiindcă s-au adăugat cele 62 de fișiere de mai sus. Fișier `.gitignore` în rădăcină:

```
bin/
obj/
doxygen/html/
*.o
*.exe
*.layout
*.depend
```

Fișierele deja urmărite nu ies singure din git după ce pui `.gitignore` — pentru ele e nevoie de `git rm -r --cached bin obj doxygen/html`. Ți-o arăt data viitoare, la calculator.

Rămase deschise din runde 3–7 — verificate prin rulare azi

#	Ce	Stare la <code>d44bbe9</code>
B14	<code>functii.h</code> nu compila	✅ închis — compilează curat
M13	patru funcții rescrise, <code>prim()</code> în loc de <code>nrPrim()</code>	✅ închis — duplicatele șterse
M14	<code>main.cpp</code> gol	✅ închis — apelează <code>sumaChenar</code>
B15	<code>functii.h:597</code> — <code>if(n%d==0);</code>	❌ neatins — <code>nrPrim</code> : 2→1, 3→1, 5→0, 7→0, 11→0, 13→0
B16	numărul de chenare ghicit ca <code>n/2</code>	❌ neatins , și a apărut a treia oară → B22
B17	<code>functii.h:478</code> — <code>return;</code> în afara lui <code>if</code>	❌ neatins — <code>afisareChenar</code> nu afișează nimic, pentru niciun chenar
B9	spirală afișează de două ori inelul degenerat	❌ neatins — pe B (5×3) tipărește 16 valori, trebuie 15; <code>a[2][1]=8</code> apare de două ori
B13	<code>produsChenar</code> fără gardă pentru chenar inexistent	❌ neatins — <code>produsChenar(B,4,5,9) = 1</code>
R1	<code>citireMatrice</code> nu e apelată de nicăieri	❌ neatins, a cincea rundă — toate soluțiile au matricea scrisă în cod
B5	ștergerea liniilor marcate, de la început spre coadă	❌ neatins
B6	<code>solutiep</code> ia ultima linie crescătoare	❌ neatins
B8	<code>solutiel</code> numără cifra maximă, nu elementul maxim	❌ neatins
M15	<code>j)</code> din fișa 2 fără soluție	❌ neatins

B17 te blochează pe fișa 3. Cerința e) cere să afișezi fiecare chenar apelând `afisareChenar`. Funcția aia nu scrie nimic pe ecran, pentru niciun chenar, de două runde. Cât timp `return;`-ul stă la `functii.h:478` în afara acoladei lui

`if`, e) nu are cum să iasă.

Before / After

#	Acum	Corect
B18	<code>main.cpp</code> include <code>functii.h</code>	include <code>solutii.h</code> (care include <code>functii.h</code>)
B19	acoladă goală la <code>functii.h:572</code>	<code>if(i1!=i2)</code> înainte ei
B19	acoladă goală la <code>functii.h:578</code>	<code>if(j1!=j2)</code> înainte ei
B20	<code>solutii.h:487</code> folosește <code>n</code> înainte de <code>:496</code>	mută <code>int n=5; int m=5;</code> sus
B20	<code>solutieC</code> / <code>solutieD</code> fără matrice și dimensiuni	declară-le la începutul funcției
B21	două funcții <code>solutieD</code> (<code>:393</code> , <code>:540</code>)	<code>solutieD</code> și <code>solutieF3D</code>
B22	<code>nrChenare=n/2;</code> în trei locuri	<code>int nr = nrChenare(n,m);</code>
B23	buclo pornește de la <code>d=3</code>	<code>if(n%2==0) return n==2;</code> înainte de buclă
B15	<code>if(n%d==0);</code> cu <code>;</code>	fără <code>;</code> — sau șterge <code>nrPrim</code> , e cod mort
B17	<code>return;</code> la <code>:478</code> , după acoladă	<code>return;</code> înăuntrul lui <code>if</code>

Q&A — verifică-ți înțelegerea (runda 8)

- `solutii.h` are 559 de linii și 16 erori, iar F9 îți scria „0 errors”. Explică-mi în două propoziții de ce. Ce anume trebuie să se întâmple cu un fișier `.h` ca să fie compilat?
- Pe matricea A (6×6), `sumaChenar` dă 874, 627, 197 — corect. Pe matricea B (5×3) dă 96 și 32, iar corect ar fi 96 și 24. Aceeași funcție, același cod. **Ce are B și n-are A?**
- În `nrElementeChenar` ai scris `if(i1==i2) return j2-j1+1;`. În `sumaChenar` ai avut `if(i1!=i2)` și ai șters-o. Ambele întreabă același lucru. Care e întrebarea, spusă în cuvinte, fără `i1` și `i2` ?

4. Ai scris azi `nrChenare` cu `(minim+1)/2`, și tot azi `nrChenare=n/2` la 230 de linii mai jos. Dacă mâine descoperim că formula corectă e altceva, în câte locuri ar trebui să o reparați acum? Și dacă ai apela funcția peste tot?
5. `isPrim(4)` întoarce 1. Bucla pornește de la `d=3`. Spune-mi ce număr nu e încercat niciodată ca împărțitor, și de ce optimizarea „merg din 2 în 2” presupune ceva ce tu n-ai scris.
-

► De rezolvat, în ordine

Primele patru sunt o singură sesiune de lucru. Nu treci la fișa 3 până nu trec toate.

1. **B18** — `main.cpp`: `#include "solutii.h"`, apoi un apel. F9. **Vei vedea erori — asta e ideea.**
2. **B20 + B21** — repară cele 16 erori: variabilele lipsă, `solutieD` dublă, șterge `solutiiC`. F9 trebuie să dea verde pe **1336** de linii, nu pe 777.
3. **B19** — pune gărzile înapoi în `sumaChenar`. Verifică: pe B (5×3), suma celor două chenare = 120.
4. **B17** — `return;` înăuntrul lui `if`. Verifică: `afisareChenar(B,5,3,0)` trebuie să scrie 12 numere, `afisareChenar(B,5,3,1)` trebuie să scrie 3.
5. **B22** — apelează `nrChenare(n,m)` în cele trei locuri cu `n/2`.
6. **B15** — șterge `nrPrim`. E moartă și e greșită.
7. **B9** — gărzile în `algoritmSpirala`. A cincea oară aceeași regulă. Verifică numărând: pe B (5×3) trebuie să iasă **15** valori.
8. **B23** — `isPrim`, și pune `exercitiiclasa.h` pe firul de include.
9. Abia apoi fișa 3: **e), f), g), h)**.

Fișa 3 e) și f) **nu pot ieși** până nu trec pașii 3 și 4 — una apelează `afisareChenar`, cealaltă `sumaChenar`. Nu sunt cerințe grele, sunt cerințe care stau pe funcțiile stricate.

Runda 7 (2026-09-23, 74eafb9 „Tema matrici”) — arhivă

Stare la `d44bbe9`: B14  · M13  · M14  · B15  · B16  (a treia apariție) · B17  · M15  · C20 . Textul complet, mai jos.

Ai atacat fișa 2 și ai rezolvat **a), b), d), e), g)** — iar **d), e) și g) sunt corecte**, verificate prin rulare, ieșire identică cu cea din fișă. E prima rundă din proiectul ăsta în care trei cerințe ies bune din prima. Ștergerea de coloane cu recalcularea lui `m` în condiția buclei (`for(int j=0;j<m;)` fără `j++` pe ramura de ștergere) e exact tiparul corect — l-ai găsit singur.

Dar `functii.h` nu compilează. 8 erori. Și motivul pentru care n-ai observat e mai important decât erorile: `main.cpp` include doar `fisa2.h`. Începem de acolo.

● B14 — `functii.h` nu compilează, iar Build (F9) îți dă verde

```
functii.h:563: error: expected ';' in 'for' statement specifier
functii.h:614,621,629,637: error: use of undeclared identifier 'prim'
functii.h:671: error: redefinition of 'stergereColoana' (prima la :56)
functii.h:688: error: redefinition of 'stergereLinie' (prima la :44)
```

Cele patru cauze, pe rând:

```
// functii.h:563
for(int j=j1; j<=j2; j++)    // <- minus in loc de egal
    ^

// functii.h:614 - ai scris functia `nrPrim`, dar o apelezi `prim`
if(prim(a[i1][j]))
```

DE CE nu te-a prins Build-ul. În C++ un header nu se compilează singur — ajunge la compilator **doar când e inclus** dintr-o unitate de compilare. Proiectul tău are o singură unitate: `main.cpp`. Iar `main.cpp` arată așa:

```
#include "fisa2.h"

int main() { }
```

Deci compilatorul vede `fisa2.h` și atât. `functii.h` și `solutii.h` — cele 1229 de linii în care ai scris toată tema — nu sunt citite niciodată. F9 compilează un fișier gol care include un header curat și îți scrie „0 errors”. Regula din fișă („Build înainte de commit”) nu e greșită, dar **poarta funcționează doar dacă ce ai scris chiar trece prin ea**. Un header pe care nu-l include nimeni e text, nu cod.

FIX: În `main.cpp`, include ce ai scris și apelează ce ai scris. Atunci F9 devine poarta reală.

● B15 — `functii.h:598` — un `;` după `if` face ca `nrPrim` să întoarcă 0 pentru orice număr ≥ 4

```
for(int d=2; d*d<=n; d++)
{
    if(n%d==0);    // <- punct si virgula aici
    return 0;
}
```

Rulat pe valori mici:

n	nrPrim(n)	corect
2	1	1
3	1	1
5	0	1
7	0	1
11	0	1
13	0	1

Efectul în cerința b): pe matricea din fișă cu `k=1`, `primeChenar` întoarce `0`, iar fișa cere `2`. Am scos punctul și virgula și am rerulat: întoarce `2`. Nimic altceva din `primeChenar` nu e greșit.

DE CE. `;` singur este o instrucțiune completă în C++ — instrucțiunea vidă. Deci `if(n%d==0);` se citește „dacă n se împarte la d, **nu face nimic**”, iar `if`-ul se termină acolo. `return 0;` de pe rândul următor nu mai aparține lui `if` — e instrucțiunea care urmează după el, și se execută necondiționat, la prima trecere prin buclă. Pentru `n=5` : `d=2`, `2*2<=5`, se intră în buclă, `if` nu face nimic, `return 0`. Gata, 5 e declarat neprim fără să fi fost testat ceva.

Indentarea nu leagă nimic în C++. Compilatorul nu vede spațiile — vede `;` și acolade. De-aia ai primit doar un *warning* („if statement has empty body”), nu o eroare: codul e perfect legal, doar că nu face ce arată.

● B16 — numărul de chenare e ghicit, nu dedus din `i1`, `j1`, `i2`, `j2` — greșit în două locuri

Fișa întreabă explicit, la a): „Când se oprește bucla peste grade? Scrie condiția pornind de la `i1`, `j1`, `i2`, `j2`, nu ghici un număr.” În ambele locuri ai ghicit un număr.

Locul 1 — `fisa2.h:66`, cerința a):

```
for(int i=0;i<=n/2;i++)
    cout<<sumaChenar(a,n,m,i)<<endl;
```

Rulat pe matricea ta 6×6:

```
874
627
197
0    <- al patrulea chenar nu exista
```

Pentru `i=3` : `i1=3, j1=3, i2=2, j2=2` . Toate buclele din `sumaChenar` pornesc de la o limită inferioară mai mare decât cea superioară, deci nu rulează niciuna, iar `s` rămâne pe `0` . **0 arată exact ca o sumă validă** — nu ai cum să-l deosebești de un chenar real plin de zerouri.

Locul 2 — `solutii.h` , `solutieChenarB()` , cerința b):

```
int nrChenare;
if(n<m) nrChenare=n/2;
else    nrChenare=m/2;
if(k>=nrChenare) cout<<"NU EXISTA";
```

Pe matricea 4x5 din fișă iese corect — din noroc, pentru că `min` e par. Pe **3x3 cu k=1** formula dă `nrChenare = 3/2 = 1` , deci `1>=1` → scrie `NU EXISTA` . Dar chenarul de grad 1 al unei matrice 3x3 **există**: `i1=j1=i2=j2=1` , un singur element, `a[1][1]` . Fișa îți spunea să testezi exact cazul ăsta.

DE CE. `n/2` e o formulă memorată, nu una dedusă — și e formula pentru *inele complete*, care pierde inelul degenerat de la mijloc. Condiția reală de existență a chenarului de grad `k` o ai deja scrisă, de două ori, în `afisareChenar` și în `algoritmSpirala` : **chenarul există cât timp `i1<=i2 && j1<=j2`** . Asta e tot. Scrisă cu `k` : `k <= n-1-k && k <= m-1-k` . Dacă vrei totuși un număr, el iese din aceeași inegalitate: `(min(n,m)+1)/2` — plus-unu-ul e fix inelul degenerat pe care `min/2` îl pierde.

FIX (locul 1):

```
for(int i=0; i<=n-1-i && i<=m-1-i; i++)
    cout<<sumaChenar(a,n,m,i)<<endl;
```

B17 — regresie: `afisareChenar` nu mai afișează nimic, niciodată — `functii.h:478`

În runda 6, B10 spunea: `afisareChenar` scrie „Nu exista” și apoi afișează oricum. Ai mutat `return` -ul. Dar l-ai pus **în afara** acoladei:

```
if(i1>i2 || j1>j2)
{
    cout<<"Nu exista ";
}
return;                // <- in afara if-ului

for(int k=j1;k<=j2;k++){    // <- de aici in jos, cod mort
    cout<<a[i1][k]<<" ";
}
...
```

Rulat:

Apel	Afișează	Ar trebui
afisareChenar(a,4,5,0)	(nimic)	1 2 3 4 5 10 15 20 19 18 17 16 11 6
afisareChenar(a,4,5,1)	(nimic)	7 8 9 14 13 12
afisareChenar(a,4,5,9)	Nu exista	Nu exista

Singurul caz care merge e cel care nu trebuia să afișeze nimic oricum.

DE CE. Un `return;` într-o funcție `void` înseamnă „ieși acum”. Pus în corpul lui `if`, iese doar pe ramura de eroare. Pus după `}`, e o instrucțiune obișnuită a funcției și se execută **de fiecare dată**, indiferent de `if`. Tot ce urmează după el devine inaccesibil — compilatorul nici nu se plânge, pentru că e cod valid, doar de neatins.

Asta e a doua oară în două runde când rezultatul depinde de **unde** cade o acoladă, nu de ce ai scris în ea (B15 e aceeași poveste cu un `;`). Când muți un `return` sau un `break`, întreabă-te mereu: e **înăuntrul** lui `if` sau **după** el? — și verifică prin rulare, nu prin citire. Diferența nu se vede la o citire rapidă.

FIX:

```
if(i1>i2 || j1>j2)
{
    cout<<"Nu exista ";
    return;
}
```

 M13 — ai rescris patru funcții pe care le aveai deja, și fix asta rupe compilarea

Funcția	O aveai la	Ai rescris-o la
stergereLinie	functii.h:44	:688
stergereColoana	functii.h:56	:671
inserareLinie	functii.h:33	:721
cifraControl	functii.h:414	:706

Cele două `stergere*` sunt **identice caracter cu caracter** cu cele vechi. Fișa le listează explicit la „Ce ai voie să folosești” și adaugă: „dacă ai nevoie de o buclă pe care ai scris-o deja în altă funcție, apelează funcția, nu copia bucla.”

Două dintre ele produc direct erorile `redefinition` din B14. Și mai e una, între fișiere: `sumaChenar` e definită și în `functii.h:555`, și în `fisa2.h:6` — deci cele două headere **nu pot fi incluse împreună** în `main.cpp`. Am încercat: `error: redefinition of 'sumaChenar'`.

Header guard-ul (`#ifndef FUNCTII_H_INCLUDED`) nu te apără aici. El oprește **includerea aceluiași fișier de două ori**, nu scrierea aceleiași funcții de două ori. Sunt două probleme diferite.

🟡 M14 — `main.cpp` e gol: nimic din temă nu e apelat

```
#include "fisa2.h"

int main()
{
}
```

În `5736b85` era `prob1();`. În `74eafb9` l-ai scos și n-ai pus nimic în loc. Consecința directă e B14: nu doar că nu testezi, dar nici nu compilezi ce ai scris. Fișa cere „fiecare cerință într-o funcție separată, apelată din `main`, și se testează pe rând” — deci `main` ar trebui să arate cam așa, cu apelurile comentate pe rând:

```
#include "solutii.h"

int main()
{
    solutieD();
}
```

🟡 M15 — `j)` are funcțiile ajutătoare, dar nu are soluția

Ai scris `coloanaCrescatoare` și `coloanaDescrescatoare` (`functii.h:733,741`), corecte amândouă. Dar nu există nicio `solutieJ()` care să le folosească, și nici `interschimbareColoane`. Deocamdată sunt cod mort.

Rămân nerezolvate: `c)`, `f)`, `h)`, `i)`, `j)`, `k)`, `l)` — le-ai marcat cu `//c?`, `//f?`, `//h)?`, `//i)??`. Ordinea în care le-aș lua: `k)` (cea mai simplă din toate, interschimbare de linii două câte două), apoi `j)` (ai deja jumătate), apoi `c)`, `f)`, `h)`, `i)`.

🟢 Cleanups

C19 — `fisa2.h:61-64` — `i1`, `j1`, `i2`, `j2` calculate în `prob1()` și nefolosite niciodată; `sumaChenar` și le calculează singură.

C20 — repo — `bin/Debug/recapitulare-matrici.exe` și `obj/Debug/main.o` sunt urmărite de git. Fișierele generate de compilator nu se comit: se schimbă la fiecare Build și umflă istoricul degeaba (ultimul commit a schimbat 78 KB de binar pentru 204 linii de cod). Proiectul `recapitulare-vacanta` are deja un `.gitignore` ; ăsta n-are.

```
bin/  
obj/  
*.o  
*.exe  
*.layout  
*.depend
```

Rămase deschise din rundele 3–6 — niciuna atinsă

Între `437f34b` și `74eafb9` : 316 linii adăugate, 2 șterse. Tot ce era deschis a rămas deschis.

#	Ce	Stare
B11	<code>int a[100][100]</code> — paranteză	✅ închis
B12	<code>produsChenar</code> era <code>void</code>	✅ închis
B10	<code>afisareChenar</code> afișa după „Nu exista”	❌ reparat greșit → B17
R1	citirea matricei dispăruse	🔗 pe jumătate — <code>citireMatrice</code> există la <code>functii.h:8</code> , dar nu e apelată de nicăieri ; toate soluțiile au matricea scrisă în cod
B13	<code>produsChenar</code> fără gardă pentru chenar inexistent	🔴 neatins — verificat: <code>produsChenar(a,4,5,9)</code> întoarce <code>1</code>
B9	spirală afișează de două ori ultimul inel	🔴 neatins — <code>algoritmSpirala</code> tot n-are <code>if(i1!=i2) / if(j1!=j2)</code> , deși <code>afisareChenar</code> le are
B5	ștergerea liniilor marcate, de la început spre coadă	🔴 neatins
B6	<code>solutieP</code> ia ultima linie crescătoare	🔴 neatins
B8	<code>solutieL</code> numără cifra maximă, nu elementul maxim	🔴 neatins

B9 e cel care mă supără cel mai tare: gărzile pentru inelul degenerat le-ai scris corect în `afisareChenar`, în `produsChenar` și acum a treia oară în `primeChenar`. În `algoritmSpirala` lipsesc în continuare. **Ai regula, dar o aplici doar unde ți-am arătat-o.**

Before / After

B15 — `nrPrim`

Acum	Corect
<code>if(n%d==0);</code> <code>....return 0;</code>	<code>if(n%d==0)</code> <code>....return 0;</code>

B16 — bucla peste grade, `fisa2.h:66`

Acum	Corect
<code>for(int i=0;i<=n/2;i++)</code>	<code>for(int i=0; i<=n-1-i && i<=m-1-i; i++)</code>

B17 — afisareChenar

Acum	Corect
<pre> if(...) { cout<<"Nu exista "; } return; </pre>	<pre> if(...) { cout<<"Nu exista "; return; } </pre>

B14 — main.cpp

Acum	Corect
<pre> #include "fisa2.h" int main() { } </pre>	<pre> #include "solutii.h" int main() { solutieD(); } </pre>

Q&A — verifică-ți înțelegerea (runda 7)

1. `functii.h` are 8 erori, dar Build (F9) îți scrie „0 errors”. Cum e posibil? Răspunde pornind de la ce anume trimite CodeBlocks la compilator când apeși F9.
2. De ce `if(n%d==0);` nu e eroare, ci doar avertisment? Ce instrucțiune C++ e acel `;` singur, și cui îi aparține `return 0;` de pe rândul următor?
3. Pe o matrice 3×3, câte chenare există? Scrie condiția de oprire folosind `i1, j1, i2, j2`, nu o împărțire la 2. Apoi verifică ce dă `n/2` și ce dă formula ta.
4. În `afisareChenar`, `return;` e după acolada lui `if`. Câte dintre cele 4 bucle de dedesubt se execută pentru `chenar=0`? De ce compilatorul nu se plânge deloc?

► De rezolvat, în ordine

1. B14 — repară `:563 ( j-j1 → j=j1 )`, `prim → nrPrim` în 4 locuri, șterge cele 4 funcții rescrise (M13). Apoi pune în `main.cpp` `#include "solutii.h"` și un apel. F9 trebuie să dea verde pe cod care chiar se compilează.

2. **B15** — scoate `;` -ul. Rulează `nrPrim` pe 2,3,5,7,11,13 și uită-te la ieșire.
3. **B17** — mută `return;` înăuntrul lui `if`. Rulează `afisareChenar` pe chenarul 0 și 1.
4. **B16** — condiția de oprire, în ambele locuri.
5. **B9** — gărzile în `algoritmSpirala`, a patra oară aceeași regulă.
6. Abia apoi cerințele noi: `k`), apoi `j`).

Nu trimite runda următoare până nu trec toate cinci. Sunt reparații de câte un rând fiecare.

Runda 6 (2026-08-05, commit `437f34b` „finish lectie 05/08”)

Ai adăugat `produsChenar`. **Începe cu asta: proiectul nu mai compilează.** Nimic nu mai rulează — nici soluțiile care mergeau ieri.

```
functii.h:502: error: subscripted value is not an array, pointer, or vector
functii.h:549: error: void function 'produsChenar' should not return a value
```

Restul runde 5 e neatins (B9, B10, M7–M9), la fel R1/B5/B6/B7/B8 din rundele 3–4.

Partea bună, și e reală: în `produsChenar` ai pus din prima gărzile `if(i1!=i2)` și `if(j1!=j2)`. Ai înțeles inelul degenerat — l-ai aplicat corect a doua oară la rând. Rămâne să-l aplici și acolo unde lipsește, în `algoritmSpirala` (B9).

🔴 B11 — `functii.h:502` — `int a[100][100]` — paranteză greșită, fișierul nu se compilează

```
void produsChenar(int a[100][100],int n , int m , int chenar)
    ^-- aici trebuia ]
```

Ai scris `[100][100]` în loc de `[100][100]`. Compilatorul citește `100[100]` ca „indexează valoarea 100”, de unde mesajul *subscripted value is not an array*. Și pentru că antetul funcției e stricat, **tot ce urmează în el e necunoscut** — de-aia primești în cascadă încă 6 erori de tipul `use of undeclared identifier 'chenar'`, `'n'`, `'m'`. Acelea nu sunt bug-uri separate: dispar toate când repara paranteza.

Ce înveți din asta: când vezi 8 erori, nu le repara pe rând de jos în sus. **Repară-o pe prima și recompilează.** O eroare de sintaxă într-un antet invalidează tot corpul funcției; numărul de erori nu spune nimic despre cât de stricat e codul.

Și: `git commit` pe cod care nu compilează. Înainte de commit, `Build` (F9 în CodeBlocks). Un commit care nu se compilează îți blochează și tot ce era bun înainte de el — acum nici `solutied`, nici spirala, nimic nu mai poate rula.

● B12 — functii.h:501,549 — funcția e void dar face return produs;

```
void produsChenar(...) // <- promite ca nu intoarce nimic
{
    int produs=1;
    ...
    return produs;      // <- dar intoarce un int
}
```

Funcția calculează un rezultat și îl întoarce, deci tipul ei nu e void, e int :

```
int produsChenar(int a[100][100],int n,int m,int chenar)
```

Mecanismul: tipul de retur e contractul funcției — ce promite că îți dă înapoi. void înseamnă „nu-ți dau nimic, doar fac ceva” (ca afisareChenar, care afișează). int înseamnă „îți dau un număr, tu decizi ce faci cu el”. produsChenar calculează, nu afișează — deci trebuie să întoarcă, iar apelantul decide dacă afișează, compară sau adună. E exact diferența dintre afisareMatrice (void) și frecventaElement (int) din același fișier — ai deja ambele tipare scrise corect.

● B13 — functii.h:502-549 — produsChenar n-are garda „chenarul nu există”

afisareChenar verifică if(i1>i2 || j1>j2). produsChenar nu verifică nimic. Rezultatele, după ce am reparat B11 și B12 și am rulat:

Apel	Ce întoarce	Ce ar trebui
produsChenar(c,5,4,2) — chenar inexistent	1	semnal de eroare / 0
produsChenar(d,2,6,1) — chenar inexistent pe 2×6	190080	semnal de eroare

Primul caz e cel perfid: întoarce 1 pentru că produs a rămas pe valoarea de inițializare și nicio buclă n-a rulat. 1 e un produs perfect valid — apelantul n-are cum să distingă „chenarul are un singur element egal cu 1” de „chenarul nu există”. Al doilea e și mai rău: întoarce un produs calculat pe celule care nu formează niciun chenar.

E a treia oară în același proiect: detectezi (sau nu) cazul special și lași codul să curgă mai departe — B10 (afisareChenar fără return), M5 (afisareMatrice în afara if/else), acum B13. Pentru o funcție care întoarce o valoare, convenția din proiectul tău e deja stabilită: -1 pentru „nu există” (îl folosești ca poz=-1 în solutiej, solutiep, solutiece).

```

if(i1>i2 || j1>j2)
{
    return -1;
}

```

M10 — `functii.h` — produsul depășește `int` chiar pe matricea ta de test

După ce reparați B11 și B12, rulați pe `v` din `afisareC` (5×4):

Apel	Rezultat
<code>produsChenar(v, 5, 4, 0)</code>	<code>0</code>
același calcul pe <code>long long</code>	<code>-8112362111612485632</code>

Chenarul exterior are 14 elemente de ordinul zecilor. `23·11·32·12·97·64·64·24·87·64·13·64·53·35` e un număr cu 22 de cifre. `int` ține până la $\sim 2 \cdot 10^9$ (10 cifre) — deci se depășește pe la al 6-lea factor, iar de acolo rezultatul e gunoi. Nici `long long` (19 cifre) nu ajunge.

Nu e o greșeală de logică, e o limită a tipului — dar înseamnă că **valoarea pe care o afișezi acum e falsă, nu doar mare**. La bac, enunțurile cu produs vin cu valori mici tocmai din motivul ăsta. Verifică pe foaia de probleme ce interval au elementele; dacă nu e garantat mic, întrebarea corectă e „ce se cere de fapt” (adesea: produsul modulo ceva, sau numărul de cifre, sau doar semnul).

M11 — `functii.h:510,517,527,537` — `if(a[i][j]!=0)` sare peste zerouri: e în enunț?

Filtrul apare în toate cele 4 bucle, deci e intenționat. Dar schimbă răspunsul: pe

```

2 0 3
4 5 6
7 8 9

```

codul tău dă `72576`; produsul *tuturor* elementelor de pe chenar e `0`.

Dacă enunțul zice „produsul elementelor **nenule** de pe chenar” — corect, e exact ce trebuie. Dacă zice doar „produsul elementelor de pe chenar” — atunci zeroul face parte din produs și filtrul e greșit. Verifică pe foaie. (Nu am enunțul lecției din 05/08 în repo — doar poza cu problemele a-r, din `exercitii/`.)

🟡 M12 — produsChenar nu e apelată de nicăieri

A treia funcție moartă din proiect, după `citireMatrice` (R1) și `solutieSpirala` (M9). `main.cpp:5` apelează tot `afisareC()`. Scrii funcții pe care nu le rulezi niciodată — de-aia B11 și B12 au ajuns în commit: dacă exista un `solutieProdusChenar()` apelat din `main`, le prindeai la primul Build.

🟢 C17 — functii.h:514-522 — acolade duplicate

```
for(int k=i1+1;k<=i2;k++)
{
    {
        // <- bloc in plus, fara rost
        if(a[k][j2]!=0)
        ...
    }
}
```

🟢 C18 — solutii.h:355-359 — indentare reparată în afisareC, dar int n=5; int m=4; (361-362) au rămas nealiniate

Rămase deschise din runda 5 (commits b441311 + 1780504)

`algoritmSpirala` și `afisareChenar`. Ambele funcționează pe matricea de test din cod (5×4). Ambele se strică pe alte dimensiuni. Și — partea interesantă — **fixul pentru bug-ul din `algoritmSpirala` există deja, scris de tine, în `afisareChenar`** (și acum și în `produsChenar`).

🔴 B9 — functii.h:456,460 — spirala afișează de două ori ultimul inel când e o singură linie sau o singură coloană

Cele patru bucle din inel merg: sus → dreapta → jos → stânga. Când inelul rămas are **o singură linie**, bucla „jos” parcurge aceeași linie ca bucla „sus”, invers. Când are **o singură coloană**, bucla „stânga” parcurge aceeași coloană ca „dreapta”.

Rulat pe o matrice 3×4 (ultimul inel = o singură linie):

```
1  2  3  4
5  6  7  8
9 10 11 12
```

Corect

Codul tău

1 2 3 4 8 12 11 10 9 5 6 7

1 2 3 4 8 12 11 10 9 5 6 7 6

Rulat pe 5×3 (ultimul inel = o singură coloană):

```
1 2 3
4 5 6
7 8 9
10 11 12
13 14 15
```

Corect

Codul tău

... 5 8 11

... 5 8 11 8

Ce se întâmplă la pasul 2 pe 3×4: `i1=1, j1=1, i2=1, j2=2`. Deci `i1==i2` — inelul are înălțime 1.

- „sus”: `a[1][1] a[1][2]` → 6 7 ✓
- „dreapta”: `k=i1+1=2` până la `i2=1` → nu intră ✓
- „jos”: `a[i2][k]` cu `i2=1` — **aceeași linie ca la pasul 1**, de la `j2-1=1` la `j1=1` → `a[1][1] = 6` ✗
- „stânga”: nu intră ✓

Fixul îl ai deja scris. Uită-te la `functii.h:486` și `functii.h:492` — în `afisareChenar` ai pus exact gărzile care lipsesc aici:

```
if(i1!=i2) { /* bucla jos */ }
if(j1!=j2) { /* bucla stanga */ }
```

Adaugă-le identic în `algoritmSpirala`. Verificat: cu gărzile, 3×4, 5×3, 5×4, 1×1 și 1×5 dau toate spirala corectă.

Mecanismul, ca să nu-l uiți: bucla „jos” e desenată ca fiind *sub* bucla „sus”. Când inelul are o singură linie, „sub” și „deasupra” sunt același lucru — geometria pe care s-a bazat algoritmul dispăre. Orice algoritm care merge pe inele concentrice are cazul ăsta: **inelul din mijloc poate fi degenerat** (o linie, o coloană, sau un singur element). Nu e un caz exotic — apare la orice matrice cu `min(n,m)` impar sau cu `n` și `m` foarte diferite.

● B10 — functii.h:474-477 — afisareChenar scrie „Nu exista” și apoi afișează oricum

```
if(i1>i2 || j1>j2)
{
    cout<<"Nu exista ";
}

// <-- nimic aici; execuția continuă în bucle
for(int k=j1;k<=j2;k++){ ... }
```

`if` -ul detectează corect că chenarul nu există, dar **nu oprește funcția**. Fără `return`, cele patru bucle rulează în continuare cu indici care nu mai descriu un chenar.

Rulat pe o matrice 2x6 cu `chenar=1` (`i1=1, i2=0` → `i1>i2`, chenarul nu există):

```
1  2  3  4  5  6
7  8  9 10 11 12
```

Corect	Codul tău
Nu exista	Nu exista 8 9 10 11 4 3 2

Deci afișează 7 elemente după ce tocmai a spus că nu există nimic de afișat. Pe matricea ta 5x4 cu `chenar=2` nimereste să iasă gol — buclele se anulează una pe alta din întâmplare. Nu e o garanție, e noroc de dimensiuni.

Fix:

```
if(i1>i2 || j1>j2)
{
    cout<<"NU EXISTA"<<endl;
    return;
}
```

De ce contează dincolo de bug: `if` fără `return` (sau fără `else`) e exact aceeași greșeală ca M5 din runda 3 — `afisareMatrice` lăsat în afara lui `if/else` în `solutiee`. E al doilea loc în care detectezi corect cazul special, scrii mesajul corect, și apoi lași codul să curgă mai departe. Regula: **un mesaj de eroare care nu e urmat de `return` sau de `else` nu oprește nimic — doar comentează**.

🟡 M7 — `functii.h:444-465` vs `467-499` — `algoritmSpirala` rescrie

`afisareChenar`

Cele două funcții fac **același lucru**: parcurg un chenar dreptunghiular în sensul acelor de ceasornic. Diferența e că una îl face pentru un `chenar` dat, iar cealaltă pentru toate, într-o buclă. Codul buclelor e copiat cuvânt cu cuvânt — cu excepția gărzilor, care există într-una și lipsesc în cealaltă. **B9 este exact prețul acestei duplicări**: ai reparat cazul degenerat într-o copie și nu în cealaltă.

Spirala scrisă *prin* chenar, fără nicio buclă duplicată:

```
void algoritmSpirala(int a[100][100],int n,int m)
{
    int nrChenare=((n<m?n:m)+1)/2;
    for(int c=0;c<nrChenare;c++)
    {
        afisareChenar(a,n,m,c);
    }
    cout<<endl;
}
```

Verificat: dă rezultat identic cu varianta cu gărzi pe 3×4 , 5×3 și 5×4 . $((\min(n,m)+1)/2)$ e numărul de inele: $5 \times 4 \rightarrow 2$ inele, $5 \times 3 \rightarrow 2$, $3 \times 4 \rightarrow 2$, $1 \times 5 \rightarrow 1$.

E al doilea caz de duplicare din proiect (vezi M6: `numerePareLinie` / `numarPareLinie`, `isLinieCrescatoare` / `linieCrescatoare`), dar ăsta e mai grav: acolo copiile erau identice, aici au divergat și una e greșită.

M8 — `functii.h:451,455,459,463` — `endl` după fiecare latură

Spirala e o **singură** secvență de numere, nu patru. Cu `endl` după fiecare latură, ieșirea pe 5×4 arată așa:

```
23 11 32 12
97 64 64 24
87 64 13
64 53 35
52 87
12 25
32
22
```

iar când o latură e goală (inel degenerat) rămâne și un rând gol. Scoate cele patru `cout<<endl;` și pune unul singur la finalul funcției. Pentru debug, un `endl` per inel (după bucla mare) e util — dar atunci pune-l acolo, o singură dată, nu de patru ori.

M9 — `solutii.h:341-363` — două soluții cu aceeași matrice hardcodată, iar `solutieSpirala` nu e apelată

`solutieSpirala` (341) și `afisareC` (353) conțin **aceeași** matrice 5×4, copiată. `main.cpp:5` apelează doar `afisareC()`, deci `solutieSpirala` nu se execută niciodată — e a doua funcție moartă din proiect, după `citireMatrice` (vezi R1). Ambele sunt simptome ale aceleiași cauze: datele stau în cod, deci fiecare soluție nouă cere o copie nouă a lor.

M4 (continuare) — matricea 5×4 nu putea prinde B9

Inelele unei matrici 5×4 sunt 5×4 și 3×2 — **niciunul degenerat**. Adică matricea pe care ai testat spirala e fix categoria de matrice pe care bug-ul nu se vede. Ca la `p`, ca la `l`, ca la `d`.

Ca să prinzi B9 îți trebuia o matrice cu `min(n,m)` impar (3×4, 5×3, 3×3) sau cu o singură linie/coloană (1×5).

**Întrebarea de pus înainte de a alege datele de test: „ce dimensiune ar rupe algoritmul?” — nu „ce dimensiune arată frumos”.

● R1 — regresie: citirea matricei a dispărut din tot proiectul (*runda a 4-a — neatins*)

`solutii.h:18,27,75,100,136,157,184,205,230,256,283,321,343,355` — acum **14** soluții cu matricea scrisă în cod (s-au adăugat două odată cu spirala).

- `citireMatrice` (`functii.h:8`) nu e apelată de nicăieri. Cod mort.
- `data.txt` e orfan.
- Enunțul problemei 1 începe cu „Sa se citeasca o matrice de maxim 100 x 200 de elemente”.
- Ca să testezi pe alte date trebuie să editezi sursa și să recompilezi — motivul pentru care B9 a rămas invizibil.

Fixul care păstrează și citirea, și testabilitatea:

```
void citireMatrice(int v[100][100], int&n, int&m, const char* fisier)
{
    ifstream read(fisier);
    read>>n>>m;
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            read>>v[i][j];
}
```

apelat cu `citireMatrice(v,n,m,"test-spirala-3x4.txt")`. Datele rămân în fișiere pe care le schimbi în 5 secunde; codul rămâne unul singur.

● B5 — functii.h:207-219 — ștergerea de la început spre coadă (runda a 3-a — neatins)

B4 (flag-urile inversate) e reparat, ordinea buclilor nu. Se vede doar cu 2+ linii de șters:

```
int a[100][100]={222,32,12},{12,13,14},{22,12,64},{32,25,333}};  
int n=4,m=3;
```

Corect	Codul tău
13 12	13 14 12 64 25 333

333 — elementul care a declanșat ștergerea — rămâne în matrice. Pas cu pas în `for(int i=0;i<n;i++)` :

- `i=0` , `delL[0]` → șterge linia 0. `n` devine 3.
- `i=1` , `i=2` → false.
- `i=3` → `3<3` fals. Bucla se oprește; `delL[3]` nu e citit niciodată.

Două lucruri se strică simultan: `n` scade sub condiția `i<n` , iar liniile rămase s-au deplasat în sus. De la coadă spre început, tot ce se deplasează e *sub* poziția curentă:

```
for(int i=n-1;i>=0;i--)  
    if(delL[i]) stergereLinie(a,n,m,i);  
for(int j=m-1;j>=0;j--)  
    if(delC[j]) stergereColoana(a,n,m,j);
```

● B6 — solutii.h:80-85 — solutiep ia ultima linie crescătoare (runda a 3-a — neatins)

Pe o matrice cu **două** linii crescătoare (0 și 2):

```
11 22 33      <- crescatoare  
99 88 77  
21 41 61      <- crescatoare  
55 55 55
```

`poz` ales = 2. Enunțul p) cere „**prima** linie ce are elementele în ordine crescătoare” → 0. Fixul e o condiție, pattern pe care îl ai deja în `solutiej:32` :

```
for(int i=0; i<n-1 && poz==-1; i++)
```

🔴 B8 — `solutii.h:233,236,239` — `solutieL` numără cifra maximă, nu elementul maxim (*runda a 2-a — neatins*)

```
90 19 13    maximul matricei = 90, apare de 2 ori pe coloana 0
90 29 23    cifra maxima = 9, apare de 3 ori pe coloana 1
31 39 33
```

Coloana aleasă = 1. Enunțul l) cere „coloana pe care se afla cele mai multe **elemente de maxim**” → 0. Cele două funcții necesare:

```
int elementMaximMatrice(int a[100][100],int n,int m)
{
    int maxim=a[0][0];
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            if(a[i][j]>maxim)
                maxim=a[i][j];
    return maxim;
}

int aparitiiElementColoana(int a[100][100],int n,int coloana,int x)
{
    int nr=0;
    for(int i=0;i<n;i++)
        if(a[i][coloana]==x)
            nr++;
    return nr;
}
```

`if(nr>=maxim)` rămâne cum e — tie-break-ul pe ultima coloană e corect.

🔄 B7 — `solutii.h:139` — `k` hardcodat (*rezolvat pe jumătate*)

`int k=3;` — nu mai e undefined behavior, dar enunțul f) spune „*k citit de la tastatura*”: `cin>>k;` .

🟡 Importante rămase din runda 4

- **M5** `solutii.h:130` — `afisareMatrice(v,n,m);` e în afara `if/else` în `solutiee`, deci afișează matricea și pe ramura „Nu exista”. Aceeași cauză ca B10. În `solutieq:308–316` ai făcut invers și corect.
- **M3** `functii.h:114` — `isNumarPp` compară `double` cu `==`. Fix în întregi: `int r=sqrt(n); return r*r==n || (r+1)*(r+1)==n;`
- **M6** `functii.h` — `numerePareLinie` (239) $\equiv$ `numarPareLinie` (320); `isLinieCrescatoare` (134) $\equiv$ `linieCrescatoare` (296).
- **M2** `main.cpp:5` — un singur apel hardcodat (acum `afisareC()`). Un `switch` peste litera problemei ar rula orice soluție fără recompilare.

🟢 Cleanups

- **C13** `functii.h:476` — `"Nu exista "` cu spațiu la final și fără `endl`; restul proiectului folosește `"NU EXISTA"` (cerut de enunț).
 - **C14** `functii.h:467` — `chenar` nu e validat pentru valori negative; `afisareChenar(a,n,m,-1)` citește `a[-1]`.
 - **C15** `solutii.h:318` — comentariu `//` gol deasupra lui `solutieR`.
 - **C16** `solutii.h:343–350,355–362` — indentare inconsistentă în cele două funcții noi (corpul nu e aliniat cu acoladele).
 - **C1** `functii.h:116` — `int&m` nefolosit în `rastoarnaColoana`. Singurul warning la `-Wall -Wextra`.
 - **C3** — `bin/Debug/*.exe`, `obj/Debug/main.o`, `.layout`, `.depend` re-comise și în ultimele două commit-uri. Un `.gitignore` cu `bin/`, `obj/`, `/*.layout`, `/*.depend` închide subiectul definitiv.
 - **C7** `functii.h:345,368,397,420` — `while(x>0)` ratează cifrele lui `0`.
 - **C8** `functii.h:229–235` — `isNumarPrim` fără ieșire timpurie.
 - **C9** `functii.h:68–86` — `treiCifre` e `bool` dar întoarce `int ok`.
 - **C12** `solutii.h:5–14` — comentariul-trace de deasupra lui `solutied` descrie o matrice care nu mai există în cod (`444 / 333 / 999`).
-

Before / After

#	Before (actual)	After (corect)
B11	<code>void produsChenar(int a[100][100],...)</code>	<code>int produsChenar(int a[100][100],...)</code>
B12	<code>void produsChenar(...)</code> <code>{ ... return produs; }</code>	<code>int produsChenar(...)</code> <code>{ ... return produs; }</code>
B13	<i>(nimic — funcția calculează direct)</i>	<code>if(...)</code> <code>{</code> <code>return -1;</code> <code>}</code>
B9	<code>for(int k=j2-1;k>=j1;k--)</code> <code>cout<<a[i2][k]<<" "</code> <code>for(int k=i2-1;k>i1;k--)</code> <code>cout<<a[k][j1]<<" "</code>	<code>if(i1!=i2)</code> <code>for(int k=j2-1;k>=j1;k--)</code> <code>cout<<a[i2][k]<<" "</code> <code>if(j1!=j2)</code> <code>for(int k=i2-1;k>i1;k--)</code> <code>cout<<a[k][j1]<<" "</code>
B10	<code>if(...)</code> <code>{</code> <code>cout<<"Nu exista ";</code> <code>}</code>	<code>if(...)</code> <code>{</code> <code>cout<<"NU EXISTA"<<endl;</code> <code>return;</code> <code>}</code>
M7	4 bucle copiate din <code>afisareChenar</code>	<code>int nrChenare=((n<m?n:m)+1)/2;</code> <code>for(int c=0;c<nrChenare;c++)</code> <code>afisareChenar(a,n,m,c);</code>
R1	<code>int v[100][100]={23,11,32,12},...};</code> <code>int n=5;</code> <code>int m=4;</code>	<code>int v[100][100],n,m;</code> <code>citireMatrice(v,n,m,"test-spirala-3x4.txt");</code>
B5	<code>for(int i=0;i<n;i++)</code> <code>if(dell[i]) stergereLinie(a,n,m,i);</code>	<code>for(int i=n-1;i>=0;i--)</code> <code>if(dell[i]) stergereLinie(a,n,m,i);</code>
B6	<code>for(int i=0; i<n-1; i++)</code>	<code>for(int i=0; i<n-1 && poz==-1; i++)</code>
B7	<code>int k=3;</code>	<code>int k;</code> <code>cout<<"k=";</code> <code>cin>>k;</code>
B8	<code>int cifraMaxima=cifraMaximaMatrice(a,n,m);</code> <code>int</code> <code>nr=aparitiiCifraColoana(a,n,j,cifraMaxima);</code>	<code>int maximElement=elementMaximMatrice(a,n,m);</code> <code>int</code> <code>nr=aparitiiElementColoana(a,n,j,maximElement);</code>

Q&A — verifică-ți înțelegerea (runda 6)

1. Compilatorul dă 8 erori în `produsChenar`, dar tu reparați **un singur caracter**. De ce dispar toate celelalte 7? Ce înseamnă asta despre ordinea în care citești erorile de compilare?
2. `afisareChenar` e `void`, `produsChenar` ar trebui să fie `int`. Care e regula prin care decizi tipul de retur al unei funcții, fără să te uiți la cod? Aplic-o pe `stergereLinie`, `frecventaElement` și `isNumarPrim` din `functii.h`.
3. `produsChenar(v,5,4,2)` întoarce `1` pentru un chenar care nu există. De ce e `1` un răspuns mai periculos decât dacă ar fi crăpat programul?
4. Pe matricea ta 5×4, produsul chenarului exterior are 22 de cifre, iar `int` ține 10. Rezultatul afișat e `0`. De ce `0` și nu „un număr mare greșit”? (indiciu: uită-te ce se întâmplă cu ultimele cifre binare la fiecare înmulțire cu un număr par)

Rămase din runda 5

5. În `afisareChenar` și în `produsChenar` ai scris `if(i1!=i2)` și `if(j1!=j2)`. În `algoritmSpirala`, aceleași bucle sunt fără gărzi. De două ori ai aplicat regula corect și o dată nu — ce îți spune asta despre codul copiat dintr-o funcție în alta?
6. Pe matricea 3×4, la al doilea inel avem `i1=1, i2=1`. Scrie pe hârtie ce afișează fiecare dintre cele 4 bucle, în ordine. La care apare al doilea `6`, și de ce bucla aia crede că e pe o linie diferită?
7. Ai testat spirala pe o matrice 5×4 și a mers. Ce au în comun 5×4, 4×4 și 6×4 — și ce au 3×4, 5×3 și 3×3 — de-ți trebuie a doua categorie ca să prinzi B9?

► De rezolvat, în ordine

1. **B11** — un `]`. Apoi **Build (F9)** și recompilează după fiecare eroare, nu după toate.
2. **B12** — `void` → `int`.
3. **M12** — scrie `solutieProdusChenar()` în `solutii.h` și apelează-o din `main.cpp`. Fără asta nu poți verifica nimic din ce urmează.
4. **B13** — `return -1;` pentru chenar inexistent.
5. **M11** — verifică pe foaia de probleme dacă e „produsul elementelor” sau „produsul elementelor nenule”.
6. **M10** — vezi ce interval de valori garantează enunțul.
7. **B10** — un `return` în `afisareChenar`. 10 secunde.
8. **B9** — cele două gărzi, copiate din `afisareChenar`. Testează pe 3×4 și 5×3 **înainte** de fix, ca să vezi duplicatul.
9. **M7** — rescrie `algoritmSpirala` ca buclă peste `afisareChenar`. Dacă faci asta, B9 dispare de la sine.
10. **M8** — scoate cele patru `endl`, pune unul la final.
11. **R1** — pune citirea înapoi, cu parametru pentru numele fișierului. Tot restul depinde de asta.
12. **M4** — fișiere de test care pot să pică: `test-spirala-3x4.txt`, unul pentru `d` (2 elemente-țintă, unul pe ultima linie), unul pentru `p` (2 linii crescătoare), unul pentru `l` (coloana maximului ≠ coloana cifrei maxime).

13. **B5, B6, B8, B7**, apoi M5/M3/M6 și cleanups.

Regulă nouă, pentru toate rundele de acum înainte: **Build** (F9) verde **înainte** de **git commit**. Un commit care nu compilează nu blochează doar codul nou — blochează tot proiectul.

Runda 4 (2026-08-05, commit 43010a9 „refacere citire”) — arhivă

Compilat cu **-Wall -Wextra**, rulate toate cele 12 soluții: toate dau rezultat corect **pe datele din cod**, dar 3 sunt greșite (**d**, **p**, **l**).

- ✓ **B4** flag-uri **delL / delC** inversate — rezolvat, verificat prin rulare.
- 🔄 **B7** **k** — nu mai e UB (**k=3**), dar enunțul cere **cin>>k**.
- **R1** citirea din fișier scoasă din toate soluțiile; **citireMatrice** devenită cod mort. → ✗ rămas.
- **B5, B6, B8** → ✗ rămase.
- **M4** datele de test nu pot infirma nimic (tabel per soluție: **d**, **p**, **l**, **h**, **m**) · **M5** · **M3** · **M6** · **M2**.
- C1, C3, C7, C8, C9, C12.

Runda 3 (2026-08-05, commit f13779a „terminare tema”) — arhivă

Tema completată (**d,e,f,g,h,j,k,l,m,p,q,r**). Corecte la rulare: **h** (fixul B3 aplicat singur), **m**, **k**, **q**, **r**, **g**, **e**.

- **B7** **solutief** — **k** folosit fără să fie citit (UB). ● **B8** **solutieL** — numără cifra maximă. ● **B4** flag-uri inversate. ● **B5** ștergere de la început. ● **B6** **solutiep** ia ultima linie.
- M5, M6, M4, M3, M2. ● C7, C8, C9, C10, C11, C3.

Runda 2 (2026-07-30, commits 0bcf19c + 4c797c3) — arhivă

- **B4** flag-urile inversate, ascuns de **444** pe diagonală la **(2,2)**. ● **B5** ștergerea de la început. ● **B6** **solutiep**.
- M3 **isNumarPp** pe **sqrt==** · M4 date de test prietenoase. ● C5 (✓ rezolvat) · C6 stil de acolade.

Runda 1 (2026-07-30, commit 99775fd) — arhivă

- **B1** **if(isNumarPp(v[i][j]==1))** — paranteza greșită. ✓ corectat.
- **B2** **i--** la **i=0** → **v[-1][j]**, UB. ✓ dispărut.
- **B3** ștergere în timpul parcurgerii; fix predat: mark-then-delete cu ștergeri de la coadă.
- M1 **sqrt** float precision · M2 **main** nu apela soluțiile. ● C1, C2, C3, C4.

